Digital KYC Verification System

Complete UI Wireframe + React + Redux + Node.js + MongoDB + OCR + Face Detection

1. Project Scope

React + Redux Toolkit + Node.js/Express + MongoDB KYC application. The 5-hour MVP covers application lookup, PAN/Aadhaar/DL verification using fictional test data, address-proof upload and OCR extraction/matching, selfie image quality and face detection, review, consent and final status. Live government verification must be implemented only through the appropriate authorized providers.

2. Complete Flow

LOGIN → DASHBOARD → APPLICATION NUMBER → CUSTOMER DETAILS → DOCUMENT SELECTION → PAN/AADHAAR/DL VERIFICATION → ADDRESS PROOF UPLOAD → OCR/DOCUMENT EXTRACTION → ADDRESS MATCH → SELFIE UPLOAD → IMAGE QUALITY → FACE DETECTION → REVIEW → CONSENT → FINAL KYC RESULT

3. UI Wireframes

Login

```
DIGITAL KYC
_____________________________

Username
[_____________________]
Password
[_____________________]
          [ LOGIN ]
```

Dashboard

```
DIGITAL KYC                          User
_____________________________________

KYC Dashboard
[ 120 Total ] [ 85 Verified ] [ 25 Pending ]

             [ START NEW KYC ]

Recent Applications
APP1001 | Rahul Kumar | VERIFIED
APP1002 | Priya       | PENDING
APP1003 | Arun        | FAILED
```

Application Number

```
KYC VERIFICATION                        Step 1/6
●—o—o—o—o—o—o
Application Number
[ APP1001_______________ ]
             [ CONTINUE ]
```

Customer Details

CUSTOMER DETAILS
Application : APP1001
Name : Rahul Kumar
DOB : 15-06-1995
Mobile : 9876543210
Email : rahul@example.com
Product : Term Insurance

[CONTINUE]

Document Selection

SELECT IDENTITY DOCUMENT
○ PAN
○ Aadhaar
○ Driving Licence

[CONTINUE]

PAN Verification

PAN VERIFICATION
PAN Number [ABCDE1234F_____]
Name [Rahul Kumar_____]
DOB [15/06/1995_____]

[VERIFY PAN]

Result:
PAN Status: ACTIVE
Name Match: YES
DOB Match : YES
✓ VERIFIED

Aadhaar Verification

AADHAAR VERIFICATION
Use the permitted/authorized verification mechanism.
[START VERIFICATION]
OTP [******]
[VERIFY]
Result: ✓ VERIFIED

Driving Licence Verification

DRIVING LICENCE VERIFICATION
Licence Number [TN01XXXXXXX_____]
Name [Rahul Kumar_____]
DOB [15/06/1995_____]
[VERIFY LICENCE]

Address Verification

ADDRESS VERIFICATION
Current Address
[25 Gandhi Street, Chennai, Tamil Nadu]
[600001_____]

Address Proof [Select Document ▼]

```
┌──────────────────────────────────────┐
│ 📄 Upload Address Proof              │
│ PDF / JPG / PNG                      │
│ Drag & Drop / Browse                 │
└──────────────────────────────────────┘

          [ EXTRACT & VERIFY ]
```

OCR Result

```
DOCUMENT EXTRACTION
Document: electricity_bill.jpg

Extracted Name    : Rahul Kumar
Extracted Address : 25 Gandhi Street, Chennai,
                    Tamil Nadu 600001
Entered Address   : 25 Gandhi Street, Chennai,
                    Tamil Nadu 600001
Match Score       : 96%

             ✓ ADDRESS VERIFIED
                [ CONTINUE ]
```

Selfie Validation

```
SELFIE IMAGE VALIDATION
┌──────────────────────────────────────┐
│             IMAGE PREVIEW            │
└──────────────────────────────────────┘
[ CHOOSE IMAGE ]

          [ VALIDATE IMAGE ]

Result:
Image Format    ✓ Valid
Image Quality   ✓ Good
Face Detected   ✓ Yes
Face Count      ✓ 1
Face Visibility ✓ Good
             ✓ SELFIE ACCEPTED
```

KYC Review

```
KYC REVIEW
Application: APP1001
Customer   : Rahul Kumar

Identity Verification : ✓ VERIFIED
Address Verification  : ✓ VERIFIED (96%)
Selfie Validation     : ✓ VERIFIED

☐ I confirm that the information is correct.

             [ SUBMIT KYC ]
```

Final Result

```
KYC VERIFIED
Application: APP1001
KYC Reference: KYC20260817001
```

```
Identity Verification ✓
Address Verification  ✓
Selfie Validation      ✓

Status: VERIFIED

       [ BACK TO DASHBOARD ]
```

4. Atomic Design

Atoms: Button, Input, Label, Select, Radio, Checkbox, DatePicker, FileUpload, Loader, ErrorMessage, StatusBadge, ProgressBar, Card, Icon.

Molecules: FormField, ApplicationNumberForm, DocumentSelector, PanForm, AadhaarForm, LicenceForm, DocumentUploader, AddressForm, VerificationStatus, SelfieUploader, ConsentBox, StepIndicator, ExtractedField.

Organisms: KycHeader, ApplicationSearch, CustomerDetails, IdentitySelection, PanVerification, AadhaarVerification, LicenceVerification, AddressVerification, DocumentExtractionResult, SelfieValidation, KycReview, KycResult, KycDashboard.

Pages: Login, Dashboard, KycStart, KycVerification, AddressVerification, SelfieValidation, KycReview, KycResult.

5. React Structure

```
src/
 ├── components/
|    ├── atoms/
|    ├── molecules/
|    └── organisms/
 ├── pages/
 ├── redux/
|    ├── store.js
|    └── slices/
|         ├── authSlice.js
|         ├── applicationSlice.js
|         ├── kycSlice.js
|         ├── verificationSlice.js
|         └── uiSlice.js
 ├── services/
|    ├── api.js
|    └── kycService.js
 ├── routes/
|    ├── AppRoutes.jsx
|    └── ProtectedRoute.jsx
```

```
├── hooks/
├── utils/
├── App.jsx
└── main.jsx
```

6. Routes

/login → /dashboard → /kyc/start → /kyc/verification → /kyc/address → /kyc/selfie → /kyc/review → /kyc/result

7. Redux State

authSlice:
user, token, role, isAuthenticated

applicationSlice:
applicationNumber, customerName, dob, mobile, email, product

kycSlice:
currentStep, documentType, identityVerification,
addressVerification, selfieValidation, consent, finalStatus

verificationSlice:
panStatus, aadhaarStatus, licenceStatus,
addressStatus, selfieStatus, loading, error

8. Backend Structure

```
server/src/
├── controllers/
|   ├── authController.js
|   ├── applicationController.js
|   ├── verificationController.js
|   └── kycController.js
├── services/
|   ├── kycService.js
|   ├── panVerificationService.js
|   ├── aadhaarVerificationService.js
|   ├── licenceVerificationService.js
|   ├── ocrService.js
|   ├── addressMatchService.js
|   └── faceValidationService.js
├── models/
|   ├── User.js
```

```
|   ├── Application.js
|   ├── KycApplication.js
|   └── VerificationMaster.js
├── routes/
├── middleware/
├── config/
└── app.js
```

9. MongoDB Collections

users: _id, username, passwordHash, role, createdAt

applications: applicationNumber, customerName, dob, mobile, email, product, status

verificationMaster: documentType, documentNumber, name, dob, address, status — fictional/test verification data

kycApplications: applicationNumber, identityVerification, addressVerification, selfieValidation, consent, kycStatus, kycReference, timestamps

10. APIs

- GET /api/applications/:applicationNumber
- POST /api/verification/pan
- POST /api/verification/aadhaar
- POST /api/verification/licence
- POST /api/verification/address
- POST /api/verification/selfie
- POST /api/kyc/submit
- GET /api/kyc/:applicationNumber

11. Document Extraction / OCR

OCR reads text from the uploaded proof. It is separate from file storage. The original file can be stored securely (for a prototype, local storage or GridFS), while extracted fields and verification results are stored in MongoDB.

Flow: React upload → multipart/form-data → Node upload middleware → store/reference original → OCR → raw text → field extraction → address normalization → comparison → VERIFIED/FAILED.

12. React Address Upload Script

```
import { useState } from "react";
import axios from "axios";

export default function AddressVerification() {
```

```
  const [file, setFile] = useState(null);
  const [address, setAddress] = useState("");
  const [result, setResult] = useState(null);

  const verify = async () => {
    const fd = new FormData();
    fd.append("proof", file);
    fd.append("enteredAddress", address);
    fd.append("applicationNumber", "APP1001");

    const { data } = await axios.post(
      "/api/verification/address", fd,
      { headers: { "Content-Type": "multipart/form-data" } }
    );
    setResult(data);
  };

  return (
    <>
      <textarea value={address}
        onChange={e => setAddress(e.target.value)} />
      <input type="file"
        accept=".pdf,.jpg,.jpeg,.png"
        onChange={e => setFile(e.target.files[0])} />
      <button onClick={verify}>Extract & Verify</button>
      {result && <p>{result.status} - {result.matchScore}%</p>}
    </>
  );
}
```

13. Node.js Address Controller

```
const ocrService = require("../services/ocrService");
const addressMatchService = require("../services/addressMatchService");

exports.verifyAddress = async (req, res) => {
  try {
    const { enteredAddress, applicationNumber } = req.body;
    if (!req.file || !enteredAddress)
      return res.status(400).json({ message: "Proof and address required" });

    const extracted = await ocrService.extractDocument(req.file.path);
    const match = addressMatchService.compareAddress(
      enteredAddress, extracted.address
    );
```

```
    res.json({
      applicationNumber,
      extractedName: extracted.name,
      extractedAddress: extracted.address,
      matchScore: match.score,
      status: match.verified ? "VERIFIED" : "FAILED"
    });
  } catch (e) {
    res.status(500).json({ message: "Document processing failed" });
  }
};
```

14. OCR Service Contract

```
async function extractDocument(filePath) {
  const rawText = await runOCR(filePath); // selected OCR library/API
  return {
    rawText,
    name: extractName(rawText),
    address: extractAddress(rawText)
  };
}

module.exports = { extractDocument };
```

Note: runOCR is intentionally a provider/library boundary. The exact OCR package/API can be plugged in without changing the controller.

15. Address Matching Script

```
function normalize(value = "") {
  return value.toLowerCase()
    .replace(/[^a-z0-9 ]/g, " ")
    .replace(/\s+/g, " ")
    .trim();
}

function compareAddress(entered, extracted) {
  const a = normalize(entered);
  const b = normalize(extracted);
  const aw = new Set(a.split(" "));
  const bw = new Set(b.split(" "));
  const common = [...aw].filter(x => bw.has(x)).length;
  const score = Math.round(
    (common / Math.max(aw.size, bw.size)) * 100
  );
```

```
  return { score, verified: score >= 80 };
}

module.exports = { compareAddress };
```

16. File Upload Middleware

```
const multer = require("multer");

const storage = multer.diskStorage({
  destination: "uploads/",
  filename: (req, file, cb) =>
    cb(null, Date.now() + "-" + file.originalname)
});

const allowed = ["image/jpeg","image/png","application/pdf"];

const fileFilter = (req, file, cb) =>
  cb(null, allowed.includes(file.mimetype));

module.exports = multer({
  storage, fileFilter,
  limits: { fileSize: 5 * 1024 * 1024 }
});
```

17. Selfie / Face Detection

The prototype does NOT perform identity face matching. It validates the uploaded image and checks for exactly one detectable face. A face-detection library/model can be used on the Node.js backend.

Flow: upload → file type/size check → decode image → quality check → face detector → count faces → exactly one face → ACCEPTED; otherwise FAILED.

18. React Selfie Upload Script

```
import { useState } from "react";
import axios from "axios";

export default function SelfieValidation() {
  const [file, setFile] = useState(null);
  const [result, setResult] = useState(null);

  const validate = async () => {
    const fd = new FormData();
    fd.append("selfie", file);
    fd.append("applicationNumber", "APP1001");
```

```
  const { data } = await axios.post(
    "/api/verification/selfie", fd,
    { headers: { "Content-Type": "multipart/form-data" } }
  );
  setResult(data);
 };

 return (
  <>
   <input type="file"
    accept="image/jpeg,image/png"
    onChange={e => setFile(e.target.files[0])} />
   <button onClick={validate}>Validate Image</button>
   {result && (
    <div>
     <p>Quality: {result.qualityScore}</p>
     <p>Face detected: {String(result.faceDetected)}</p>
     <p>Face count: {result.faceCount}</p>
     <p>Status: {result.status}</p>
    </div>
   )}
  </>
 );
}
```

19. Node.js Face Validation Controller

```
const faceService =
 require("../services/faceValidationService");

exports.verifySelfie = async (req, res) => {
 try {
  if (!req.file)
   return res.status(400).json({ message: "Selfie required" });

  const result = await faceService.validate(req.file.path);

  res.json({
   qualityScore: result.qualityScore,
   faceDetected: result.faceDetected,
   faceCount: result.faceCount,
   status: result.verified ? "VERIFIED" : "FAILED"
  });
 } catch (e) {
  res.status(500).json({ message: "Selfie processing failed" });
```

```
  }
};
```

20. Face Validation Service Contract

```
async function validate(filePath) {
  const qualityScore = await calculateQuality(filePath);
  const faces = await detectFaces(filePath);

  return {
    qualityScore,
    faceDetected: faces.length > 0,
    faceCount: faces.length,
    verified: qualityScore >= 70 && faces.length === 1
  };
}

module.exports = { validate };
```

calculateQuality and detectFaces are replaceable implementation points for the selected image-processing/face-detection library.

21. Mock PAN Verification Script

```
const VerificationMaster =
  require("../models/VerificationMaster");

async function verifyPan({ pan, name, dob }) {
  const record = await VerificationMaster.findOne({
    documentType: "PAN",
    documentNumber: pan
  });

  if (!record)
    return { verified: false, message: "PAN not found" };

  const nameMatch =
    record.name.trim().toLowerCase() === name.trim().toLowerCase();

  const dobMatch = record.dob === dob;

  return {
    verified: record.status === "ACTIVE" && nameMatch && dobMatch,
    nameMatch,
    dobMatch,
    status: record.status
```

```
  };
}

module.exports = { verifyPan };
```

22. Final Submit Logic

POST /api/kyc/submit

Validate:
1. Identity verification = VERIFIED
2. Address verification = VERIFIED
3. Selfie validation = VERIFIED
4. Consent = true
5. Application exists

If all pass:
 save kycStatus = VERIFIED
 generate KYC reference
Else:
 save FAILED or PENDING_REVIEW

23. Final Architecture

```
React
 ↓
Atomic UI (Atoms → Molecules → Organisms → Pages)
 ↓
Redux Toolkit
 ↓
Axios
 ↓
Node.js + Express
 ↓
Controllers → Services
  ├── PAN/Aadhaar/DL verification
  ├── OCR/document extraction
  ├── Address matching
  └── Face detection/image quality
 ↓
MongoDB
  ├── applications
  ├── kycApplications
  ├── verificationMaster
  └── users
```

Production:
Replace mock verification services with appropriate authorized external providers.

24. 5-Hour Build Plan

1. Hour 1: React setup, routing, Atomic Design components, Login and Dashboard.
2. Hour 2: Application lookup, customer details, document selection, Redux.
3. Hour 3: Node/Express APIs, MongoDB models, mock PAN/Aadhaar/DL verification.
4. Hour 4: File upload, OCR/document extraction, address normalization and matching.
5. Hour 5: Selfie upload, image quality + face detection, Review, Submit and Result.

25. Production/Security Notes

- Never expose external provider secrets in React; keep them in Node.js environment variables.
- Use fictional/test identity data for the prototype.
- Use only appropriate authorized mechanisms/providers for real PAN/Aadhaar/DL verification.
- Treat uploaded identity documents as sensitive and use secure storage, access control and retention policies.
- Face detection is not the same as identity face matching.
- OCR extraction can be imperfect; production systems should support document-specific parsing and manual review.